Im doing Game implemented via LLM and prompt engineering

The document you attached contains detailed instructions and requirements for a project involving a game platform with a specific focus on "Game implemented via LLM and prompt engineering." Here's a concise overview relevant to your interest in the game implemented using Large Language Models LLMs and prompt engineering:

Game Implemented via LLM and Prompt Engineering

- The initial game to be developed is a simple two-player game like TicTacToe or Connect Four.
- The game should be refactored and rebuilt in Python using LLMs and prompt engineering methods.
- Proper software engineering practices must be applied in the refactoring and rebuilding of the game.
- The design should account for AI integration and gameplay logging.
- Minimum requirements include:
  - Using LLMs to refactor the existing game code.
  - Writing unit tests to test the game's functionality.
  - Applying prompt engineering techniques learned in class.
  - Maintaining a Prompt Log documenting prompt strategies and iterations.
- Deliverables:
  - Link to the original game files.
  - Rebuilt Python game code.
  - Prompt Log in Markdown or PDF format.
  - Unit test suite.

Additional Key Points

- The game state must be comprehensive to describe the current game situation unambiguously, determine legal actions, evaluate state, generate next states, and adapt to game specifics.
- Gameplay logging must capture detailed data used for analytics, AI training, and insights.
- The AI player acts autonomously using strategies like Monte Carlo Tree Search, Minimax, or heuristic-based approaches.
- The project involves an AI lifecycle with dataset generation, machine learning model training, monitoring, and possibly continuous improvement.
- A chatbot component can provide platform and game support using LLM-based RetrievalAugmented Generation RAG.
- The entire system should expose AI capabilities via REST APIs and be deployable as Docker containers.

This sets a framework for developing a game using LLMs and prompt engineering with detailed requirements on software engineering quality, testing, AI integration, and data logging you should follow. If you want, the next step could be to dive deeper into any of these aspects such as prompt engineering strategies, AI player design, game state modeling, or logging implementation. Just let me know what you want to focus on next! [1]

⁂

Rewrite this TicTacToe game logic written in JavaScript to Python, including board initialization, current player management, and resetting the game. file:/C/Users/khale/Desktop/KDG/year3/integration5/banditgames-ai/game_engine/game.js

Here is a Python version of the TicTacToe game logic you provided, including board initialization, current player management, checking for win/tie, and game reset. The logic is wrapped in a class for modularity:

```
class TicTacToe :
    def __init__ (self):
        self.players = [ 'X', 'O']
        self.current_player =  self.players[^ 2_0]
        self.board = [ ''] * 9
        self.someone_won =  False
        self.winning_combinations = [
            [ 0, 1, 2], [3, 4, 5], [6, 7, 8],
            [ 0, 3, 6], [1, 4, 7], [2, 5, 8],
            [ 0, 4, 8], [2, 4, 6]
        ]
        self.end_message =  "X's turn!"

    def make_move (self, index ):
        if self.someone_won:
            return
        if self.board[index] !=  '':
            return
        self.board[index] =  self.current_player
```

```
        if self.check_win():
            self.someone_won = True
            self.end_message = f"Game over! {self.current_player} wins!"
            return
        if self.check_tie():
            self.someone_won = True
            self.end_message = "Game is tied!"
            return
        self.switch_player()
        self.end_message = f"{self.current_player}'s turn!"

    def check_win(self):
        for a, b, c in self.winning_combinations:
            if self.board[a] == self.current_player and \
               self.board[b] == self.current_player and \
               self.board[c] == self.current_player:
                return True
        return False

    def check_tie(self):
        return all(square != '' for square in self.board)

    def switch_player(self):
        self.current_player = self.players[^2_1] if self.current_player == self.players[

    def restart_game(self):
        self.someone_won = False
        self.board = [''] * 9
        self.current_player = self.players[^2_0]
        self.end_message = "X's turn!"

# Example usage
game = TicTacToe()
print(game.end_message)  # X's turn!

game.make_move(0)  # Player X moves
print(game.board)
print(game.end_message)

game.make_move(1)  # Player O moves
print(game.board)
print(game.end_message)

game.restart_game()
print(game.board)
print(game.end_message)
```

This class captures the same features from the JavaScript logic: a 3 3 board stored in a list, two players 'X' and 'O', toggling turns, checking for wins and ties, and resetting the game. The `end_message` conveys the state to the user similar to the UI text in your JS version.

Let me know if you want this extended with additional features or adapted for integration with your Python-based LLM/game platform system.

⁂

Convert the move validation and application functions into Python methods inside a TicTacToe class with appropriate error handling.

Here is an updated Python version of the TicTacToe class that includes move validation and application as methods with appropriate error handling using exceptions:

```
class InvalidMoveError(Exception):
    """Exception raised for invalid moves in TicTacToe."""
pass

class TicTacToe:     def __init__(self):
self.players = ['X', 'O']
self.current_player = self.players[0]
self.board = [''] * 9
self.someone_won = False
self.winning_combinations = [
[0, 1, 2], [3, 4, 5], [6, 7, 8],
            [0, 3, 6], [1, 4, 7], [2, 5, 8],
            [0, 4, 8], [2, 4, 6]
        ]         self.end_message =
f"{self.current_player}'s turn!"

    def validate_move(self, index):
        if not 0 <= index < 9:
            raise InvalidMoveError("Move index out of bounds.")
if self.someone_won:
            raise InvalidMoveError("Game already ended.")
if self.board[index] != '':
            raise InvalidMoveError("Square already occupied.")

    def apply_move(self, index):
try:
self.validate_move(index)
except InvalidMoveError as e:
print(f"Invalid move: {e}")
return False
                self.board[index] =
self.current_player         if
self.check_win():
            self.someone_won = True
            self.end_message = f"Game over! {self.current_player} wins!"
elif self.check_tie():
            self.someone_won = True
self.end_message = "Game is tied!"         else:
            self.switch_player()
            self.end_message = f"{self.current_player}'s turn!"
return True

    def check_win(self):         for a, b, c in
self.winning_combinations:
```

```
            if self.board[a] ==  self.current_player  and \
               self.board[b] ==  self.current_player  and \
               self.board[c] ==  self.current_player:
                return  True
        return  False

    def check_tie (self):
        return  all(square !=  '' for square  in self.board)

    def switch_player (self):
        self.current_player =  self.players[ 1] if self.current_player ==  self.players[ 0] e

    def restart_game (self):
        self.someone_won =  False
        self.board = [ ''] * 9
        self.current_player =  self.players[ 0]
        self.end_message =  f"{self.current_player  }'s turn!"

# Example of usage
game = TicTacToe()
print(game.end_message )

move_successful = game.apply_move(  0)
print(game.board )
print(game.end_message )

move_successful = game.apply_move(  0)  # Invalid move (already occupied)
print(game.board )
print(game.end_message )

game.restart_game()
print(game.board )
print(game.end_message )
```

This code introduces an `InvalidMoveError` exception for invalid moves. The `validate_move` method raises this if the move is out of bounds, the game ended, or the square is occupied. The `apply_move` method calls `validate_move` and handles exceptions by printing an error message, returning `False` without applying the move. It returns `True` if the move was successful and updates game state accordingly.

Let me know if you'd like further refinement or integration tips.

Provide idiomatic Python functions to check for a win or tie on the TicTacToe board.

Here are two idiomatic Python functions that check for a win or a tie in a TicTacToe game, assuming the board is represented as a list of 9 elements:

```
def check_win(board, player):
    """Check if the given player has won."""
winning_combinations = [
        [0, 1, 2], [3, 4, 5], [6, 7, 8],  # Rows
        [0, 3, 6], [1, 4, 7], [2, 5, 8],  # Columns
        [0, 4, 8], [2, 4, 6]              # Diagonals
```

```
    ]     return any(all(board[pos] == player for pos in combo) for combo in
winning_combinatio

def check_tie(board):
    """Check if the game is a tie (no empty spaces and no winner)."""
return all(square != '' for square in board)
```

Usage

- `check_win(board, 'X')` or `check_win(board, 'O')` will return `True` if the specified player has a winning line. • `check_tie(board)` returns `True` if the board is full and no player has won, indicating a tie.

Example

```
board = ['X', 'X', 'X', '', '', '', '', '', '']
print(check_win(board, 'X'))  # True
print(check_tie(board))       # False
```

These functions are concise and leverage list comprehensions with the `any()` and `all()` functions for clarity and idiomatic Python style.

Write Python methods to switch players and handle the game flow after each move.

Here are Python methods that switch players and handle the game flow after each move inside a TicTacToe class context:

```
class TicTacToe:
    def __init__(self):
        self.players = ['X', 'O']
self.current_player = self.players[0]
self.board = [''] * 9
self.someone_won = False
        self.end_message = f"{self.current_player}'s turn!"
        def
switch_player(self):
        """Toggle the current player."""
self.current_player = (
            self.players[1] if self.current_player == self.players[0] else
self.players[0
        )

    def handle_move(self, index):
        """
        Handle game updates after a player makes a move.
        Assumes the move is valid and already applied to the board.
        """
        if self.check_win():
            self.someone_won = True
            self.end_message = f"Game over! {self.current_player} wins!"
elif self.check_tie():
```

```
            self.someone_won = True
self.end_message = "Game is tied!"         else:
            self.switch_player()             self.end_message
= f"{self.current_player}'s turn!"

    def check_win(self):
winning_combinations = [
            [0, 1, 2], [3, 4, 5], [6, 7, 8],
            [0, 3, 6], [1, 4, 7], [2, 5, 8],
            [0, 4, 8], [2, 4, 6]
        ]
return any(
            all(self.board[pos] == self.current_player for pos in combo)
for combo in winning_combinations
        )

    def check_tie(self):         return all(square !=
'' for square in self.board)
```

Explanation:

- `switch_player`: Simply toggles the current player between 'X' and 'O'.
- `handle_move`: Updates the game state after a move, checking if the move caused a win or tie, sets appropriate status, or switches players and updates the message accordingly.
- The `check_win` and `check_tie` functions are used inside `handle_move` to determine the game status.

This pattern keeps the game flow logic clean and manageable, assuming move validation and application happen before calling `handle_move`.

Add docstrings and comments to the Python TicTacToe class for clarity and maintainability.

Here is the Python TicTacToe class with detailed docstrings and comments added for clarity and maintainability:

```
class InvalidMoveError (Exception ):
    """Exception raised for invalid moves in TicTacToe."""
    pass

class TicTacToe :
    """
    A class to represent a TicTacToe game.

    Attributes:
        players (list[str]): List of player symbols, typically ['X', 'O'].
```

```
        current_player (str): The player whose turn it is currently.
        board (list[str]): Represents the 3x3 TicTacToe board as a list of 9 strings.
        someone_won (bool): Flag indicating if the game has a
winner.         end_message (str): Message indicating the current
game status.     """         def __init__(self):
        """Initialize a new game of TicTacToe."""
self.players = ['X', 'O']
        self.current_player = self.players[0]
        self.board = [''] * 9  # Empty board represented by empty
strings         self.someone_won = False         self.end_message =
f"{self.current_player}'s turn!"

    def validate_move(self, index):
"""
        Validate a move to ensure it's within bounds, the game is ongoing, and the
target                Args:
            index (int): The board index (0-8) where the player wants to
move.
        Raises:
            InvalidMoveError: If the move is invalid.
        """         if not 0
<= index < 9:
            raise InvalidMoveError("Move index out of
bounds.")         if self.someone_won:             raise
InvalidMoveError("Game already ended.")         if
self.board[index] != '':             raise
InvalidMoveError("Square already occupied.")

    def apply_move(self, index):
        """
        Attempt to apply a valid move for the current
player.                 Args:
            index (int): The board index where the player intends to move.
        Returns:
            bool: True if move was successful, False otherwise.


        Side effects:
            Modifies the board, switches players, updates game status and end message.
        """
try:
            self.validate_move(index)
except InvalidMoveError as e:
print(f"Invalid move: {e}")
return False


        self.board[index] = self.current_player  # Apply the move


        self.handle_move()  # Handle post-move checks and
updates         return True     def handle_move(self):
```

```
        """
        Handle game state updates after a move has been applied.
        Check for win or tie, update flags and messages, or switch players.
        """         if
self.check_win():
            self.someone_won = True
            self.end_message = f"Game over! {self.current_player} wins!"
elif self.check_tie():
            self.someone_won = True
self.end_message = "Game is tied!"         else:
self.switch_player()             self.end_message =
f"{self.current_player}'s turn!"

    def check_win(self):
        """
        Check if the current player has won the game.

Returns:
            bool: True if current player has a winning combination, False otherwise.
        """
        winning_combinations = [
            [0, 1, 2], [3, 4, 5], [6, 7, 8],  # Horizontal
            [0, 3, 6], [1, 4, 7], [2, 5, 8],  # Vertical
            [0, 4, 8], [2, 4, 6]              # Diagonals
        ]
return any(
            all(self.board[pos] == self.current_player for pos in combo)
for combo in winning_combinations
        )

    def check_tie(self):
        """
        Check if the game has ended in a tie (no empty squares left).

Returns:
            bool: True if all squares are filled and no winner, False otherwise.
        """         return all(square != '' for square
in self.board)

    def switch_player(self):
        """
        Switch the current player to the other player.
        """
        self.current_player = (
            self.players[1] if self.current_player == self.players[0] else
self.players[0
        )

    def restart_game(self):
        """
        Reset the game state to start a new game.
        """
self.someone_won = False
self.board = [''] * 9
        self.current_player = self.players[0]
self.end_message = f"{self.current_player}'s turn!"
```

This version clearly documents class responsibilities, method arguments, return values, side effects, and error conditions with standard Python docstrings and inline comments for maintainability. Let me know if you need additional features or further explanations.

Design a Python method to serialize and return the current game state (board layout, current player, game status) as a JSON-compatible dict."

Here is a Python method added to the TicTacToe class that serializes and returns the current game state as a JSON-compatible dictionary:

```
import json

class TicTacToe:
    # ... (existing methods) ...

    def serialize_state(self):
        """
        Serialize the current game state into a JSON-compatible dictionary.

        Returns:             dict: A dictionary containing the board
layout, current player,                   whether the game has ended, and
the current status message.
        """
return {
            'board': self.board.copy(),            # Board layout as list of strings
            'current_player': self.current_player, # Whose turn it is
            'game_over': self.someone_won,         # Whether game ended
            'status_message': self.end_message     # Informational
message         }

# Usage example
game = TicTacToe()
state_dict = game.serialize_state()
json_string = json.dumps(state_dict, indent=2)  # Convert to JSON string if needed
print(json_string)
```

Notes:

- The returned dictionary is JSON-friendly and can be converted to a JSON string with `json.dumps`.
- The board is copied to avoid accidental modifications of the internal state from outside.
- This method encapsulates all the essential info to restore or inspect the current game status for UIs, logging, or interoperability.

Implement functions to log each move with details like player, position, timestamp, and resulting board state.

Here is an implementation of move logging functionality as part of the TicTacToe class. It records each move with player, position, timestamp, and resulting board state:

```
from datetime import datetime

class TicTacToe:     def __init__(self):
self.players = ['X', 'O']
self.current_player = self.players[0]
self.board = [''] * 9
self.someone_won = False
```

```
        self.end_message = f"{self.current_player}'s turn!"
self.move_log = []  # Initialize empty move log list

    def log_move(self, position):
        """
        Log the details of the current move.
Args:             position (int): The board index of the
move made.
        """
        move_details = {
            'player': self.current_player,
            'position': position,
            'timestamp': datetime.utcnow().isoformat() + 'Z',  # UTC ISO 8601 format
'board_state': self.board.copy()
        }
        self.move_log.append(move_details)
        def apply_move(self, index):
try:
self.validate_move(index)
except InvalidMoveError as e:
print(f"Invalid move: {e}")
return False


        self.board[index] = self.current_player  # Apply move
self.log_move(index)  # Log the move immediately after applying

        self.handle_move()  # Update game status
return True

    # Other methods (validate_move, handle_move, check_win, check_tie, switch_player,
res
```

How it works:

- `move_log` is a class attribute list storing move records.
- `log_move` is called right after a move is applied to capture who played, where, and when, plus the board state after the move.
- The timestamp is in ISO 8601 UTC format for universal standardization. `board.copy()` is used to
- store a snapshot of the board state at that move.

You can access `game.move_log` after or during the game to inspect the full history of moves played for analytics, debugging, or replay purposes. Let me know if you want code for exporting or formatting this log.

Write methods to keep a history of all moves played during the game inside the TicTacToe class.

Here is an enhanced version of the TicTacToe class with methods to keep a full history of all moves played during the game, stored internally in a list:

```
from datetime  import  datetime

class  TicTacToe :
    def  __init__ (self):
        self.players = [ 'X', 'O']
        self.current_player =  self.players[ 0]
        self.board = [ ''] * 9
        self.someone_won =  False
        self.end_message =  f"{self.current_player  }'s turn!"
        self.move_history = []   # Store all moves with details

    def  log_move (self, position ):
        """
        Record the details of the current move into move history.

        Args:
            position (int): Board position index for the move.
        """
        move_entry = {
             'player' : self.current_player,
             'position' : position,
             'timestamp' : datetime.utcnow().isoformat() +   'Z',
             'board_state' : self.board.copy()
        }
        self.move_history.append(move_entry)

    def  apply_move (self, index ):
        """
        Apply a move and record it if valid.

        Args:
            index (int): Board position to place the current player's marker.


        Returns:
            bool: True if move was valid and applied, False otherwise.
        """
try:
            self.validate_move(index)
except InvalidMoveError as e:
print(f"Invalid move: {e}")
return False


        self.board[index] = self.current_player
self.log_move(index)  # Log after applying move
self.handle_move()         return True

    # Include or import other methods (validate_move, handle_move, switch_player,
check_w
```

Summary:

- `move_history` list keeps each move's details as a dictionary.
- `log_move()` captures player, position, timestamp, and board snapshot immediately after applying the move.

- All moves played this game session can be accessed via the `move_history` attribute for replay, analytics, or debugging.

This method ensures a comprehensive, chronological record of the game progress inside the class. Let me know if you want to add move undo/redo or export capabilities too.

Show how to export the full gameplay log suitable for replay or analysis.

Here is a Python method to export the full gameplay log from the TicTacToe class as a JSON formatted string, suitable for replay or analysis:

```
import json

class TicTacToe:
    # ... (existing methods)

    def export_move_history(self):
        """
        Export the full move history as a JSON-formatted string.

        Returns:
            str: JSON string containing the list of all moves and their details.
        """
        return json.dumps(self.move_history, indent=2)

# Usage example
game = TicTacToe()
# Assume several moves have been applied here...

json_log = game.export_move_history()
print(json_log)
```

Explanation:

- `export_move_history` converts the internally stored `move_history` list (which includes player, position, timestamp, and board state snapshots) into a pretty-printed JSON string.
- The JSON string can be saved to a file, sent over a network, or used by other systems for replay or detailed analysis of the game steps.

If you want, this method can be extended to support exporting to a file directly or to other formats. Let me know if you want those additions.

Create a Python FastAPI service that exposes an endpoint /game/state which returns the current TicTacToe board state and game status as JSON.

Here is a simple Python FastAPI service exposing a `/game/state` endpoint that returns the current TicTacToe board state and game status as JSON. It assumes you have the TicTacToe class with a `serialize_state()` method as defined earlier:

```
from fastapi import FastAPI
from pydantic import BaseModel
import uvicorn

# Assume TicTacToe class with serialize_state() is defined or imported above

app = FastAPI()
game = TicTacToe()

class GameState(BaseModel):
    board: list[str]
    current_player: str
    game_over: bool
    status_message: str

@app.get("/game/state", response_model=GameState)
def get_game_state():
    """
    Returns the current TicTacToe game state as JSON.
    """
    state = game.serialize_state()
    return GameState(
        board=state['board'],
        current_player=state['current_player'],
        game_over=state['game_over'],
        status_message=state['status_message']
    )



# To run the server, use:
# uvicorn script_name:app --reload
```

Notes:

- The `GameState` Pydantic model defines the JSON schema for the endpoint response.
- The global `game` instance holds the current TicTacToe game state.
- `serialize_state()` returns the dictionary representing board, player, game over status, and message.
- Start this FastAPI server with `uvicorn your_script:app --reload` and access `/game/state` to get the current game state.

Let me know if you want endpoints for making moves, restarting the game, or logging history!

Write a POST endpoint /game/move in FastAPI that accepts a move position, validates and applies the move

using the TicTacToe class, and returns the updated game state and any result message.

Here is a FastAPI POST endpoint `/game/move` that accepts a JSON payload with the move position, validates and applies it to the TicTacToe game instance, and returns the updated game state along with a result message:

```
from fastapi  import  FastAPI, HTTPException
from pydantic  import  BaseModel, Field
from typing  import  Optional

app = FastAPI()
game = TicTacToe()

class  MoveRequest (BaseModel ):
    position:  int = Field(..., ge= 0, le=8, description= "Board position to place the move

class  GameStateResponse  (BaseModel ):
    board:  list[str]
    current_player:   str
    game_over:  bool
    status_message:   str
    move_result:   Optional [str] = None

@app.post( "/game/move" , response_model=GameStateResponse   )
def  make_move (move: MoveRequest ):
    """
    Accepts a move position, validates and applies it if valid,
    then returns the updated game state and a message describing the move result.
    """
    if game.someone_won:
        raise HTTPException(status_code=400, detail="Game already finished.")

    success = game.apply_move(move.position)     if not success:         raise
HTTPException(status_code=400, detail="Invalid move. Square occupied or out

    state = game.serialize_state()
return GameStateResponse(
board=state['board'],
        current_player=state['current_player'],
game_over=state['game_over'],
status_message=state['status_message'],
move_result="Move applied successfully."     )

# Include the /game/state GET endpoint and TicTacToe class definition from previous
code
```

Explanation:

- `MoveRequest` validates the input to accept an integer position from 0 to 8.
- The endpoint raises HTTP 400 errors if the game is finished or the move is invalid.
- On success, it applies the move, retrieves the updated game state, and includes a success message.
- This complements the `/game/state` GET endpoint and enables clients to submit moves interactively.

You can extend with endpoints to restart the game, retrieve move history, etc. Let me know if you want those as well.

should i put it with game state in a controller.py

Yes, organizing your FastAPI route handlers (endpoints) like `/game/state` and `/game/move` inside a dedicated `controller.py` file is a good practice. This approach helps keep your project clean and maintainable by separating the web API layer from your core game logic.

Recommended project structure example:

```
/your_project
|
 ├──controller.py          # FastAPI endpoints (controllers)
 ├──game.py                # TicTacToe class and game logic
 ├──main.py                # FastAPI app instance, include router from controller.py
 ├──requirements.txt
 └── ...
```

Why use this structure?

- **Separation of concerns:** Game logic (in `game.py`) is independent of how the API is exposed.
- **Maintainability:** Easier to navigate and update code.
- **Scalability:** You can add more controllers for other features (e.g., user auth, stats) without clutter.

Example `controller.py` snippet:

```
from fastapi import APIRouter, HTTPException from models import TicTacToe,
MoveRequest, GameStateResponse  # import from your modules

router = APIRouter()
game = TicTacToe()

@router.get("/game/state",
response_model=GameStateResponse) def get_game_state():
return game.serialize_state()

@router.post("/game/move",
response_model=GameStateResponse) def make_move(move:
MoveRequest):     if game.someone_won:          raise
HTTPException(400, "Game already finished.")     if not
game.apply_move(move.position):
        raise    HTTPException(400,    "Invalid

move.")      return game.serialize_state()
```

Then in `main.py`:

```
from fastapi  import  FastAPI
from controller  import  router   # import router from controller module

app = FastAPI()
app.include_router(router)
```

This modular design aligns well with professional API development and will scale better as you add features like move history export or game restart endpoints.

Let me know if you want a more detailed example or help setting this up!

Write Python unit tests for the TicTacToe class methods: board initialization, move validation, win/tie checking, and player switching.

Here is an example Python unit test suite for the TicTacToe class methods using the built-in `unittest` framework. It tests board initialization, move validation, win/tie checking, and player switching:

```
import unittest from
datetime import datetime

# Import TicTacToe and InvalidMoveError from your
module # from your_module import TicTacToe,
InvalidMoveError

class
TestTicTacToe(unittest.TestCase):
def setUp(self):         self.game =
TicTacToe()

    def test_board_initialization(self):
        # Board should be empty strings and current player should be 'X'
self.assertEqual(self.game.board, ['']*9)
self.assertEqual(self.game.current_player, 'X')
self.assertFalse(self.game.someone_won)

    def test_validate_move_valid(self):
        # Valid moves should pass without
Exception         try:
self.game.validate_move(0)
        except InvalidMoveError:             self.fail("validate_move
raised InvalidMoveError unexpectedly!")

    def test_validate_move_out_of_bounds(self):
with self.assertRaises(InvalidMoveError):
            self.game.validate_move(-1)
with self.assertRaises(InvalidMoveError):
self.game.validate_move(9)

    def test_validate_move_occupied(self):
        # Apply a move and then validating same position should raise
error         self.game.board[0] = 'X'         with
```

```
self.assertRaises(InvalidMoveError):
self.game.validate_move(0)

    def test_validate_move_game_ended(self):
self.game.someone_won = True         with
self.assertRaises(InvalidMoveError):
self.game.validate_move(1)

    def test_check_win(self):
        # Setup a winning condition for X first row
self.game.board = ['X', 'X', 'X', '', '', '', '', '', '']
self.assertTrue(self.game.check_win())
        # Not a winning state
        self.game.board = ['X', '', 'X', '', 'O', '', '', '', '']
self.assertFalse(self.game.check_win())

    def test_check_tie(self):
        # Full board with no winner is a tie
        self.game.board = ['X', 'O', 'X', 'O', 'X', 'O', 'O', 'X', 'O']
self.game.someone_won = False
self.assertTrue(self.game.check_tie())
        # Not full board is no tie
self.game.board[0] = ''
self.assertFalse(self.game.check_tie())

    def test_switch_player(self):
self.assertEqual(self.game.current_player, 'X')
self.game.switch_player()
        self.assertEqual(self.game.current_player, 'O')
self.game.switch_player()
self.assertEqual(self.game.current_player, 'X')

if __name__ == '__main__':
unittest.main()
```

How to run:

- Save this as, for example, `test_tictactoe.py`.
- Adjust import statements to import your `TicTacToe` and `InvalidMoveError` classes properly. Run
- using: `python -m unittest test_tictactoe.py` These tests cover:
- Initial state correctness.
- Move validation boundary and error cases.
- Detecting wins and ties.
- Player switch correctness.

You can extend this suite with more detailed tests for `apply_move`, `handle_move`, move logging, serialization, and restart functionality.

Let me know if you want examples for those!